Ders 6 Lab Çalışma Özeti

Ders 6 Lab Çalışma Özeti

Genel Konular

- **Veritabanı Kurulumu (pgAdmin)**
  - CompanyDB veritabanının oluşturulması
  - Şema dosyası ile tabloların oluşturulması (CREATE TABLE komutları)
  - Veri dosyası ile tablolara INSERT INTO ile kayıt eklenmesi
  - 6 tablolu Company veritabanı: Employee, Department, Project, Works_On, Dependent, Dept_Locations
- **Kısıtlar (Constraints)**
  - Primary Key (Birincil Anahtar)
    - Tek sütunlu primary key tanımlama (örn: SSN, team_number)
    - Çoklu sütun primary key tanımlama (örn: team_number + SSN birlikte)
    - PRIMARY KEY tanımı: tabloda her kayıt için benzersiz ve tanımlayıcı olmalı
    - Örnek: Student ID, TC Kimlik Numarası gibi alanlar primary key'dir
  - Foreign Key (Yabancı Anahtar)
    - Başka bir tablonun primary key'ine referans veren sütun
    - REFERENCES ile hangi tablonun hangi sütununa referans verdiği belirtilir
    - ON DELETE CASCADE: ana tablodan kayıt silindiğinde ilişki tablosundaki ilgili kayıtların da otomatik silinmesi
  - CHECK Kısıtı
    - Sütun değerleri üzerinde koşul kontrolü (örn: playtime < 13)
  - CONSTRAINT isimlendirmesi
    - İsim verilerek tanımlanırsa: CONSTRAINT isim PRIMARY KEY ...
    - İsim verilmezse otomatik isimlendirme: tablo_adı_pk, tablo_adı_sütun_adı_fk
    - ALTER TABLE ... DROP CONSTRAINT ile kısıt kaldırma
- **View (Görünüm)**
  - Sorgulara verilen isim, sanal tablo
  - CREATE VIEWview_adi AS SELECT ... ile oluşturulur
  - Sık kullanılan sorguları kaydederek pratiklik sağlar
  - Dinamiktir: tabloda değişiklik yapıldığında view otomatik güncellenir
  - SELECT * FROM view_adi ile çağrılır
- **Sequence (Sayı Dizisi)**
  - Otomatik artan numaralı değerler üretmek için kullanılır
  - CREATE SEQUENCE ile oluşturulur
  - Özellikleri: MINVALUE, MAXVALUE, INCREMENT BY, START WITH, CYCLE/NO CYCLE
  - nextval('sequence_adi') ile bir sonraki değer alınır
  - Tablolara kayıt eklerken otomatik ID atamak için kullanılır
- **Küme İşlemleri (Set Operations)**
  - UNION (Birleşim): İki sorgunun tüm kayıt değerlerini birleştirir (VEYA)
  - INTERSECT (Kesişim): İki sorgunun ortak kayıt değerlerini getirir (HEM HE)
  - EXCEPT (Fark): Birinci sorgudaki ama ikincide olmayan kayıtları getirir (DIŞLAMA)
  - Önemli: Her iki sorgudaki SELECT kısımları aynı sütunları içermeli
- **İç İçe Sorgular (Nested Queries)**
  - EXISTS: Bir sorgu sonucunda kayıt olup olmadığını kontrol eder
  - NOT EXISTS: Belirtilen koşulu sağlayan kaydın olmaması durumu
  - IN: Bir değer listesinde olup olmadığını kontrol eder

  - Aynı tablonun birden fazla kez kullanılması gerekebilir (örn: E1 ve E2 ile Employee tablosunun iki kere kullanılması)
  - SüperSSN (yönetici kimlik numarası) ile SSL (çalışan kimlik numarası) eşleştirme
- **Aggregate Fonksiyonlar**
  - SUM: Toplam değer hesaplama
  - MIN: Minimum değer bulma
  - MAX: Maksimum değer bulma
  - AVG: Ortalama değer hesaplama
  - COUNT: Kayıt sayısını bulma (COUNT(*) ile tüm satırlar)

Hocanın Özellikle Vurguladığı Kısımlar

- **Foreign Key ve ON DELETE CASCADE ilişkisi**
  - ON DELETE CASCADE eklenmezse ana tablodan kayıt silinmeye çalışıldığında hata alınır
  - Cascade eklendiğinde ana tablodan silinen kayıtla ilişkili tüm kayıtlar otomatik silinir
  - İlişki tablosundan kayıt silmek her zaman serbesttir, ana tabloya etki etmez
- **Yönetici (Manager) kavramında dikkat edilmesi gerekenler**
  - Bir kişinin yönetici olması için illa departman yöneticisi olması gerekmez
  - Bir kişi 3-5 kişilik bir grubun da yöneticisi olabilir
  - SüperSSN alanındaki herkes departman tablosunda yönetici olarak geçmeyebilir
  - Hatalı yaklaşım: Department tablosundaki yönetici üzerinden gidip bulmaya çalışmak
- **Küme işlemlerinde sütun uyumu**
  - UNION, INTERSECT, EXCEPT işlemlerinde her iki sorgunun SELECT kısımları aynı sütunları seçmeli
  - Farklı sütun isimleri veya sayıları kullanılamaz
  - Aynı isim ve soyisimden iki farklı çalışan varsa, SELECT'te sadece bu sütunlar isteniyorsa sadece bir kez gösterir; farklı sütunlar da istenirse (örn: SSN) her iki kayıt da gösterilir
- **Employee tablosunun birden fazla kez kullanılması**
  - İç içe sorgularda aynı tablonun farklı alias'larla kullanılması gerekir (E, E2 gibi)
  - SüperSSN ile SSL eşleştirmek için iki Employee tablosu birlikte kullanılır
  - Bir tablonun kendi içinde ilişkisel sorgular yazılırken bu zorunluluktur

Kısa Tekrar Notları

- Primary Key: tabloda her kayıt için benzersiz, NULL olamaz, tek veya çoklu sütun olabilir
- Foreign Key: başka bir tablonun primary key'ine referans, NULL olabilir
- ON DELETE CASCADE: ana tablodan silme yapıldığında ilişkili kayıtları da siler
- CONSTRAINT ismi vermek: ileride ALTER TABLE ile kısıtı kaldırmayı kolaylaştırır
- View: sanal tablo, sorgulara isim verme, dinamik olarak güncellenir
- Sequence: otomatik artan numaralar üretir, nextval() ile değer alınır
- UNION: iki sorgunun birleşimi (VEYA), INTERSECT: kesişim (HEM VE), EXCEPT: fark (DIŞLAMA)
- EXISTS/NOT EXISTS: iç içe sorgularda varlık kontrolü
- Aggregate fonksiyonlar: SUM, MIN, MAX, AVG, COUNT ile istatistiksel hesaplamalar

Detaylı Açıklamalar

- **CompanyDB Veritabanı Yapısı:** Employee (çalışanlar), Department (departmanlar), Project (projeler), Works_On (çalışan-proje ilişkisi), Dependent (çalışan akrabaları), Dept_Locations (departman lokasyonları) olmak üzere 6 tablodan oluşur. Employee tablosunda SSN (Social Security Number) birincil anahtardır ve her çalışanı tanımlar. Bu alanada birçok tabloda referans olarak kullanılır.
- **Primary Key Oluşturma Yöntemleri:** İki temel yöntem vardır. Birincisi, CONSTRAINT anahtar kelimesi ile isim vererek: `CONSTRAINT pk_team PRIMARY KEY (team_number)`. Bu yöntemde isim verildiği için ileride `ALTER TABLE team DROP CONSTRAINT pk_team` ile kolayca kaldırılabilir. İkinci yöntem, sütun tanımı yanında doğrudan PRIMARY KEY

yazmaktır: `team_number INTEGER PRIMARY KEY`. Bu durumda sistem otomatik olarak `team_alter_pk` gibi bir isim oluşturur.

- **Foreign Key ve Referans İlişkileri:** Team_Employee ilişki tablosunda, team_number sütunu team tablosunun team_number sütununa, SSN sütunu ise employee tablosunun SSN sütununa referans verir. `ON DELETE CASCADE` eklendiğinde, team tablosundan bir takım silindiğinde o takıma ait tüm Team_Employee kayıtları da otomatik silinir. Aynı şekilde employee tablosundan bir çalışan silindiğinde de ilişkili kayıtlar silinir. Cascade eklenmezse silme işlemi hata verir ve izin verilmez.
- **CHECK Kısıtı Örneği:** `CONSTRAINT chk_playtime CHECK (playtime < 13)` ifadesiyle, bir çalışanın bir takımda 12 haftadan uzun oynayamayacağı kuralı tanımlanır. Bu tür kısıtlar veri doğruluğunu sağlamak için kullanılır.
- **View Oluşturma ve Kullanımı:** `CREATE VIEW maaslar AS SELECT fname, lname, salary FROM employee WHERE salary BETWEEN 20000 AND 40000` komutuyla, maaşı 20.000-40.000 arasında olan çalışanların ad, soyad ve maaş bilgilerini içeren sanal bir tablo oluşturulur. Daha sonra `SELECT * FROM maaslar` ile bu view çağrılabilir. Employee tablosunda maaş güncellendiğinde view'daki veriler de otomatik olarak güncellenir.
- **Sequence Oluşturma ve Kullanımı:** `CREATE SEQUENCE sec START WITH 9 MINVALUE 9 MAXVALUE 99 INCREMENT BY 1 NO CYCLE` komutuyla 9'dan başlayıp 99'a kadar birer birer artan bir sayı dizisi oluşturulur. `nextval('sec')` ifadesi her çağrıldığında bir sonraki değeri döndürür. Yeni kayıt eklerken `INSERT INTO employee (fname, ssn) VALUES ('Ali', nextval('sec'))` şeklinde kullanılarak otomatik ID atanabilir.
- **Küme İşlemleri Detayı:** Operating System projesinde VE Software departmanında çalışanları bulmak için iki ayrı sorgu yazılır ve aralarına INTERSECT koyularak kesişim kümesi alınır. Aynı şekilde "Operating System'de VEYA Software'de çalışanlar" denildiğinde UNION, "Operating System'de çalışıp Software'de çalışmayanlar" denildiğinde EXCEPT kullanılır. Bu işlemler kümeler mantığıyla çalışır.
- **NOT EXISTS ile Yönetici Dışlama:** "Hiçbir şekilde yönetici olmayan çalışanları bul" sorusunda, Department tablosundaki MGR_SSN ve Employee tablosundaki SuperSSN alanlarında geçen tüm yönetici kimlik numaraları dışlanır. Bunun için iki ayrı NOT EXISTS bloğu kullanılır: biri departman yöneticilerini, diğeri ise çalışanların yöneticilerini dışlamak için.
- **IN ile İç İçe Sorgu:** "John adlı çalışanın çalıştığı departmanın adı" sorusunda, önce John'ın DNO'su (departman numarası) bulunur, sonra bu numara department tablosunda eşleştirilerek DNAME (departman adı) getirilir. `SELECT dname FROM department WHERE dnumber IN (SELECT dno FROM employee WHERE fname = 'John')` şeklinde yazılır.
- **Aggregate Fonksiyon Kullanımı:** Sales departmanında kaç kişi çalıştığını ve toplam/minimum/maksimum/ortalama maaşları bulmak için SUM, MIN, MAX, AVG ve COUNT fonksiyonları kullanılır. `SELECT SUM(salary), MIN(salary), MAX(salary), AVG(salary), COUNT(*) FROM employee, department WHERE dname = 'Sales' AND dnumber = dno` sorgusuyla tek satırlık bir sonuç elde edilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.